

NestJS Interview Questions & Answers

Welcome to the NestJS Interview Questions and Answers repository!

- This repository aims to be a comprehensive resource for NestJS developers preparing for interviews. Whether you're a beginner or an experienced developer, this collection of questions and answers is designed to help you brush up on your NestJS knowledge and excel in interviews.

Click ☆if you like the project and incase you're interested in contributing to this project. Before you dive in, please take a moment to review our guidelines.

Important Links

- [Code of Conduct](#): I expect everyone participating in this project to follow the code of conduct. Make sure you understand and adhere to these guidelines.
- [Contributing Guidelines](#): Before contributing, please read the contributing guidelines. They provide information on how to submit questions, report issues, and more.

Follow me [@gasangw](#).

Table of Contents

No.	Questions
1	What is Nestjs?
2	Who developed NestJS? Why did they develop NestJS?
3	When was NestJS first released?
4	How can you install NestJS and set up a new project on your machine?
5	What's the difference between NestJS and Angular?
6	Is it possible to use other languages like C++, Ruby or Python with NestJS? If yes, then how?
7	What are the main components of a NestJS application?
8	How to declare a class as a controller in Nest.js
9	Can you explain how to use decorators in a NestJS controller?
10	How can you use route parameters in a NestJS controller?
11	What is the role of the @Body() decorator?
12	What is an interceptor in the context of NestJS?
13	What are pipes in the context of NestJS?

No.	Questions
14	What are guards in the context of NestJS?
15	What are middlewares in the context of NestJS?
16	Explain the concept of Dependency Injection in NestJS. How does it help in building modular and testable applications?
17	What's the difference between @injectable() and @inject() decorators?
18	How does the Nest logger differ from the standard console.log() and when would you prefer one over the other?
19	What is the difference between interceptors and middleware?
20	What testing frameworks work best with NestJS?
21	Explain the purpose of DTOs (Data Transfer Objects) in NestJS.
22	How can you handle asynchronous operations in NestJS, and what is the role of the Promise object?
23	Explain the purpose of the @InjectRepository() decorator in NestJS.
24	Explain the purpose of the @nestjs/jwt package in NestJS?
25	Discuss how tokens are used for authorization in an API. What is the difference between authentication and authorization, and how are these processes implemented with tokens?
26	Why is it important for tokens to have an expiration time? How can you implement token expiration in NestJS, and what role do refresh tokens play in maintaining user sessions?
27	Describe the mechanism for a token refresh in NestJS. How can you implement an automatic token refresh strategy to maintain user sessions?
28	How does NestJS support authentication and authorization?
29	What is the difference between Provider and Services in Nestjs, can we have a provider without an injectable decorator, Give examples?
30	What are custom providers and how do they differ from standard Providers in Nest.js?
31	How can you generate API documentation using Swagger in NestJS? Discuss the importance of

No.	Questions
	<u>documenting your API and how it benefits developers?</u>
32	<u>Explain the purpose of the @nestjs/swagger ApiProperty(), ApiOperation() decorators?</u>
33	<u>Explain the purpose of the Dockerfile in a NestJS application, and how it facilitates containerization?</u>
34	<u>How can you use Docker Compose with NestJS, and what is its role in a multi-container setup?</u>
35	<u>What is the purpose of the @nestjs/passport package, and how does it facilitate authentication in NestJS?</u>
36	<u>How can you handle file uploads in NestJS, and what is the role of the Multer library?</u>
37	<u>How does NestJS handle database interactions, and what are the supported databases?</u>
38	<u>What is Circular dependency (dependency cycle) in Nestjs, and how can they be fixed?</u>
39	<u>How can you handle errors in NestJS?</u>
40	<u>How does NestJS handle CORS (Cross-Origin Resource Sharing)?</u>
41	<u>Explain the purpose of the ExecutionContext in NestJS Middleware?</u>
42	<u>How can you implement soft deletes in NestJS using TypeORM, and why might soft deletes be preferred over hard deletes?</u>
43	<u>Explain the concept of environment variables in NestJS, and how can they be utilized for configuration management?</u>
44	<u>What is the role of migration scripts in TypeORM, and how can you create and run migrations in a NestJS application?</u>
45	<u>What is the purpose of ExecutionContext in NestJS?</u>
46	<u>What is the purpose of the @Res() decorator in NestJS controllers?</u>
47	<u>Explain the various Modules in NestJS?</u>
48	<u>How can you secure your NestJS application?</u>
49	<u>What is the entry file of NestJs application?</u>
50	<u>What is the difference between dependency injection and inversion of control (IoC)?</u>
51	<u>How can you implement Caching in NestJS?</u>

No.	Questions
52	Explain the purpose of the Dependency Inversion Principle (DIP) in NestJS?
53	How can you schedule tasks in NestJS?
54	How can you handle database transactions in NestJS, and why are transactions important in certain scenarios?
55	How can you implement versioning in NestJS APIs?
56	Explain the purpose of the <code>@nestjs/graphql Resolver</code> and <code>@nestjs/graphql Scalar</code> decorators, and how they relate to GraphQL in NestJS?
57	Explain the concept of Serialization and Deserialization in NestJS?
58	Explain the role of NestJS middleware in the context of Microservices and provide a scenario where middleware is beneficial in a Microservices setup?
59	Discuss the different types of coupling, such as tight coupling and loose coupling, and provide examples of how NestJS modules contribute to achieving loose coupling in a modularized application?
60	How does NestJS support Server-Sent Events (SSE), and what are the primary advantages of using SSE for real-time communication in web applications?

Answers

1. What is Nestjs?

Nest(NestJS) is a framework for building efficient, scalable Node.js server side applications. It uses progressive JavaScript and its built with and fully supports Typescript.

1. Who developed NestJS? Why did they develop NestJS?

NestJS was developed by Kamil Myśliwiec, who is a Polish software engineer. He developed NestJS to address the lack of a consistent structure in Node.js applications and to bring powerful features of frameworks like Angular to the server-side.

1. When was NestJS first released?

NestJS was first released on October 5, 2016.

1. How can you install NestJS and set up a new project on your machine?

To install NestJS on your machine, you need to have Node.js and npm (Node Package Manager) installed. Once you have those, you can install the NestJS CLI (Command Line Interface) globally on your machine using the following command:

```
$ npm i -g @nestjs/cli
```

This command installs the NestJS CLI globally, which allows you to use the nest command from anywhere on your machine. With the NestJS CLI, you can create new projects using

```
$ nest new project-name
```

and after creating a project you can generate NestJS modules, services, etc.

```
$ nest generate module users
```

running

```
$ nest g resource users
```

will several files that work together to handle CRUD (Create, Read, Update, Delete) operations for a particular entity, in this case, "users". It will generate: A controller for handling HTTP requests (e.g., users.controller.ts)

A service for business logic (e.g., users.service.ts)

A module to encapsulate the resource (e.g., users.module.ts)

If you choose to generate a REST API, it will also generate DTO (Data Transfer Object) classes for handling input data (e.g., create-user.dto.ts, update-user.dto.ts) If you choose to generate a GraphQL API, it will also generate a resolver (e.g., users.resolver.ts)

1. What's the difference between NestJS and Angular?

Angular is a framework for building client-side applications and It provides a way to organize your frontend code using components, modules, services, etc.

while NestJS is a framework for building server-side applications. NestJS is built on top of TypeScript and Express, and it aims to provide a more robust and scalable architecture for enterprise-level applications. However It's heavily inspired by Angular and shares similar concepts like modules, decorators, and dependency injection.

1. Is it possible to use other languages like C++, Ruby or Python with NestJS? If yes, then how?

Yes, it is possible to use other languages with NestJS. NestJS is language agnostic, meaning that it can work with any language that can compile to JavaScript.

As for Python, Ruby, or other languages, they can't be used directly with NestJS because NestJS relies on the Node.js runtime, which executes JavaScript. As for Python, Ruby, or

other languages, they can't be used directly with NestJS because NestJS relies on the Node.js runtime, which executes JavaScript.

However, you can certainly build separate services in Python, Ruby, or any other language, and have them communicate with your NestJS application via HTTP, gRPC, or any other communication protocol. This is a common pattern in microservices architecture.

1. What are the main components of a NestJS application?

Modules: Modules are a way of organizing related components into a single block. They provide a way to structure your application.

Controllers: Controllers are responsible for handling incoming requests and returning responses to the client. Controllers organize routes and handle HTTP requests that come to those routes.

Services: services are responsible for business logic and interacting with data sources. They can be injected into controllers or other services, promoting code reusability and separation of concerns.

1. How to declare a class as a controller in Nest.js

In Nest.js we can declare a class as a controller by using the `@Controller()` decorator. Here is a basic example.

```
import { Controller, Get } from "@nestjs/common";

@Controller("example")
class ExampleController {
  @Get()
  getHello(): string {
    return "Hello world!";
  }
}
```

In this example the `ExampleController` is a controller class. `@Controller('example')` decorator tells Nest.js that this class is a controller that should handle requests to the `example` route. The `@Get()` decorator on the `getHello` method indicates that this method should handle HTTP GET requests.

[↑ Back to Top](#)

1. Can you explain how to use decorators in a NestJS controller?

Before explaining how to use decorators, let me explain more about what are decorators:

decorators are special functions that are prefixed with an `@` symbol and can be attached to classes, methods, or properties. They are used to add metadata, methods, properties, or observe the behavior of the classes, methods, or properties they are attached to.

NestJS provides several built-in decorators, and you can also create custom decorators. Here

are some examples of built-in decorators in NestJS:

2. **Class decorators** like `@Controller()`, `@Module()`, `@Injectable()`, etc. These are used to annotate classes.
3. **Method decorators** like `@Get()`, `@Post()`, `@Put()`, etc. These are used to annotate methods within controller classes to handle specific routes.
4. **Parameter decorators** like `@Req()`, `@Res()`, `@Body()`, etc. These are used to annotate parameters within route handling methods.
5. **Property decorators** like `@Inject()`. These are used to annotate properties within classes.
6. **Custom decorators**. You can create your own decorators to handle common tasks across your application.

For example, below is how **method decorators** are used to handle GET, POST, PUT, DELETE requests respectively.

```
import {
  Controller,
  Get,
  Param,
  Body,
  Post,
  Patch,
  Delete,
} from "@nestjs/common";

@Controller("cats")
export class CatsController {
  @Get()
  findAll(): string {
    return "This action returns all cats";
  }

  @Get(":id")
  findOne(@Param("id") id: number): string {
    return `This action returns a cat with the provided id`;
  }

  @Post()
  create(@Body() body: any): string {
    return `This action returns the body of the cat`;
  }

  @Patch("id")
  update(@Param("id") id: number, @Body() body: any): string {
    return `This action updates the body of the cat`;
  }

  @Delete("id")
  remove(@Param("id") id: number): string {
    return `This action removes a cat`;
  }
}
```

[↑ Back to Top](#)

1. How can you use route parameters in a NestJS controller?

Route parameters in a NestJS controller can be accessed using the `@Param()` decorator in the controller methods.

```
@Patch('id')
update(@Param('id') id: number, @Body() body: any ): string {
  return `This action updates the body of the cat`;
}
```

[↑ Back to Top](#)

1. What is the role of the `@Body()` decorator?

The `@Body()` decorator in NestJS is used to extract the entire body of the incoming HTTP request. It's commonly used in methods that handle POST and PUT requests where data is sent in the body of the request.

For example, if you have a method in your controller to create a new user, you might use the `@Body()` decorator to get the user data from the request:

```
@Post()
create(@Body() createUserDto: CreateUserDto) {
  return this.userService.create(createUserDto);
}
```

In this example, `createUserDto` is an object that contains the data sent in the request body. The `@Body()` decorator automatically parses the JSON request body and assigns it to the `createUserDto` parameter.

[↑ Back to Top](#)

1. What is an interceptor in the context of NestJS?

An interceptor is a class annotated with the `@Injectable()` decorator and implements the `NestInterceptor` interface.

An Interceptor is function that can be used to intercept incoming requests to a NestJS application and perform some sort of manipulation before the request is handled by the route handler. This can be useful for things like logging, authentication and so on.

An Interceptor has a set of useful capabilities which are inspired by the Aspect Oriented Programming (AOP) technique.

Aspect Oriented Programming is a programming paradigm that aims to increase modularity by allowing the separation of cross-cutting concern.

Interceptors make it possible to:

Binding extra logic before / after method execution: An Interceptor can execute logic before and after a method is executed. This can be useful for tasks like logging, transforming the result of a method, or handling errors.

Transforming the result returned from a function: An Interceptor can

transform the response returned from a method. For example, you could use an interceptor to transform all responses to have a specific format.

Handling errors: An Interceptor can also handle errors thrown within your application. This can be useful for logging or transforming errors before they're sent to the client. Here is an example of an interceptor that logs the user interactions as shown in the [Nestjs document](#)

```
import {
  Injectable,
  NestInterceptor,
  ExecutionContext,
  CallHandler,
} from "@nestjs/common";
import { Observable } from "rxjs";
import { tap } from "rxjs/operators";

@Injectable()
export class LoggingInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    console.log("Before...");

    const now = Date.now();
    return next
      .handle()
      .pipe(tap(() => console.log(`After... ${Date.now() - now}ms`)));
  }
}
```

[↑ Back to Top](#)

1. What are pipes in the context of NestJS?

Pipes are a feature of NestJS that allows for validation or transformation of data before it is passed to a route handler and can either return the arguments as is, modify them, or throw an exception. A pipe is a class annotated with the `@Injectable()` decorator, which implements the `PipeTransform` interface

Validation: This is the most common use case for pipes. They can be used to validate incoming data to ensure it matches a certain data schema. If the data is invalid, the pipe can throw an exception to prevent the route handler from being executed.

Transformation: Pipes can transform incoming data into a desired format. For example, you might want to automatically convert incoming strings to numbers, or convert an entity's ID into the entity itself. Here's an example of a simple pipe that validates and transforms an incoming string into a number:

```
import {
  PipeTransform,
  Injectable,
  ArgumentMetadata,
  BadRequestException,
} from "@nestjs/common";

@Injectable()
export class ParseIntPipe implements PipeTransform<string, number> {
  transform(value: string, metadata: ArgumentMetadata): number {
```

```

    const val = parseInt(value, 10);
    if (isNaN(val)) {
        throw new BadRequestException("Validation failed");
    }
    return val;
}
}

```

This `ParseIntPipe` is therefore used in this way

```

@Get()
async findOne(@Query('id', ParseIntPipe) id: number) {
    return this.userService.findOne(id);
}

```

Nest comes with nine pipes available out-of-the-box: **ParseIntPipe**: This pipe transforms an incoming string into an integer. If the string cannot be parsed into an integer, it throws an exception. As shown above.

ValidationPipe: This pipe validates that the incoming request body matches a specific DTO (Data Transfer Object). If the data is invalid, it throws an exception.

ParseFloatPipe: Similar to **ParseIntPipe**, but transforms an incoming string into a float.

ParseBoolPipe: This pipe transforms an incoming string into a boolean. It accepts 'true' and 'false' as valid boolean strings.

ParseArrayPipe: This pipe transforms a comma-separated string into an array. It can also validate the items in the array if you provide a validation schema.

ParseUUIDPipe: This pipe validates that an incoming string is a valid UUID. If the string is not a valid UUID, it throws an exception.

ParseEnumPipe: This pipe validates that an incoming string is a valid value of a specific TypeScript enum.

DefaultValuePipe: This pipe provides a default value if the incoming value is undefined or null.

ParseFilePipe: This pipe is used to handle file uploads or parse file data in some way.

[**↑ Back to Top**](#)

1. What are guards in the context of NestJS?

Guards are functions that have a single responsibility. They determine whether a given request will be handled by the route handler or not, depending on certain conditions (like permissions, roles, ACLs, etc.) present at run-time. For example, you might use a guard to check if a user is logged in before allowing them to access a route.

A guard is a class annotated with the `@Injectable()` decorator, which implements the `CanActivate` interface. This interface should return a boolean or a Promise that resolves to a boolean. If it returns true, the request proceeds to the route handler. If it returns false,

the request is denied and the route handler is not executed. Here's an example of a basic guard:

```
import { Injectable, CanActivate, ExecutionContext } from
"@nestjs/common";

@Injectable()
export class AuthGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean | Promise<boolean> {
    const request = context.switchToHttp().getRequest();
    // Add your authentication logic here
    // For example, check if a valid JWT token exists in the request
    headers
    return true; // or false if the request should be denied
  }
}
```

In this example, AuthGuard allows all requests to proceed. In a real application, you would add your authentication logic in the canActivate method.

[↑ Back to Top](#)

1. What are middlewares in the context of NestJS?

Middleware is a function which is called before the route handler. Middleware functions have access to the request and response objects, and the next () middleware function in the application's request-response cycle.

The next middleware function is commonly denoted by a variable named next.

Middlewares can be used for a variety of purposes, such as logging, authentication, and authorization. These functions are used to execute any code, make changes to the request and the response objects, end the request-response cycle, or call the next middleware function in the stack.

Middlewares can perform the following tasks:

7. Execute any code.
8. Make changes to the request and the response objects.
9. End the request-response cycle.
10. Call the next middleware function in the stack. If the current middleware function does not end the request-response cycle, it must call next() to pass control to the next middleware function. Otherwise, the request will be left hanging. On how to create a middleware check the [Nestjs Documentation](#)

[↑ Back to Top](#)

1. Explain the concept of Dependency Injection in NestJS. How does it help in building modular and testable applications?

Dependency Injection (DI) is a design pattern in which a class receives its dependencies from external sources rather than creating them itself. This pattern is fundamental to the way NestJS is designed. dependency injection involves letting the framework manage the

creation and injection of dependencies into the components (controllers, services, and more) as needed.

This is achieved through decorators, providers, and the NestJS IoC (Inversion of Control) container. Here is an example:

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class AppService {
  getHello(): string {
    return 'Hello World!';
  }
}

import { Controller, Get } from '@nestjs/common';
import { AppService } from './app.service';

@Controller()
export class AppController {
  constructor(private appService: AppService) {}

  @Get()
  getHello(): string {
    return this.appService.getHello();
  }
}
```

In this example, `AppService` is a provider that is injected into `AppController` through the constructor. When `AppController` is created, NestJS automatically creates an instance of `AppService` and passes it to the constructor.

DI enables us to build modular and testable applications in several ways:

Modularity: By injecting dependencies, you can easily swap out one implementation for another. This is useful when you want to change the behavior of parts of your application without changing the classes that use them.

Testability: DI makes it easy to unit test your classes, as you can inject mock versions of dependencies for testing purposes.

Separation of Concerns: Each class focuses on its own behavior, delegating the behavior of its dependencies to the classes that implement those dependencies. This leads to cleaner, more readable code.

[↑ Back to Top](#)

1. What's the difference between `@injectable()` and `@inject()` decorators?

`@Injectable()`: This decorator marks a class as a provider that can be managed by the NestJS dependency injection system. It means that NestJS will create an instance of this class and can inject it where it's needed.

It's typically used on services, which can then be injected into controllers or other services.

```
@Injectable()
export class CatsService {
  // ...
}
```

@Inject(): This decorator is used inside a class to inject a dependency. It's used in the constructor of a class to specify a dependency that should be injected.

If you're injecting a class provider, you don't need to use **@Inject()** because TypeScript's reflection system can infer the type. But if you're injecting a non-class provider (like a value or a factory), or if you're working in JavaScript, you need to use **@Inject()** to tell NestJS what to inject.

```
export class CatsController {
  constructor(@Inject('CatsService') private catsService: CatsService)
  {}
}
```

[**↑ Back to Top**](#)

1. How does the Nest logger differ from the standard console.log() and when would you prefer one over the other?

The NestJS Logger provides additional features compared to `console.log()`. It includes context information, supports log levels such as (log, fatal, error, warn, debug, and verbose), and can be customized.

Use `console.log()` for quick debugging or simple logging needs.

Use NestJS Logger when you need more control over your logs, such as in larger or production applications. It helps in filtering logs based on levels and provides context, making it easier to trace the source of the log.

On how the Nest logger is implemented checkout [Nestjs Documentation](#)

[**↑ Back to Top**](#)

1. What is the difference between interceptors and middleware?

In NestJS, both `interceptors` and `middleware` can be used to add extra logic before or after HTTP requests. However, they have some key differences:

Interceptors: have a more comprehensive scope than middleware. They can be used with both HTTP requests and other types of transport like WebSockets and microservices.

Interceptors can also manipulate the response sent back to the client, for example by transforming the response object, adding extra headers, or changing the status code. They can also be used to implement performance tracking, logging, caching, etc.

Middleware: in NestJS is similar to Express middleware. It's specific to the HTTP request-response cycle and can't be used with other types of transport. Middleware functions have access to the request and response objects, and they can end the request-response cycle

or call the next middleware function in the stack.

They are useful for tasks like logging, error handling, or validating request data.

In general, if you're working with HTTP requests and you need to add logic that doesn't modify the response sent to the client, middleware can be a good choice. If you need to add logic that applies to other types of transport such as WebSockets and microservices, or if you need to modify the response, use an interceptor.

[↑ Back to Top](#)

1. What testing frameworks work best with NestJS?

NestJS is a Node.js framework, so any testing framework that works with Node.js will work with NestJS. Some popular options include **Jest**, **Mocha**, and **Jasmine**. NestJS is built with testing in mind and it comes with its own testing module called `@nestjs/testing`. This module provides utilities for testing, such as a testing module and HTTP testing utilities.

[↑ Back to Top](#)

1. Explain the purpose of DTOs (Data Transfer Objects) in NestJS.

DTOs are used to define the structure of data exchanged between different layers of an application. They define the shape of data for a specific operation, such as creating, updating, or returning data.

DTOs serve several purposes which include: **Validation**: With the `class-validator` package, you can add validation rules to the fields in your DTOs. NestJS can automatically validate incoming requests against these rules and return an error if the request is invalid.

Documentation: DTOs provide a clear model of what the data should look like. This can be helpful for other developers, and for tools like Swagger to automatically generate API documentation.

Type Safety: DTOs provide type safety in TypeScript, which can help catch errors at compile time.

```
import { IsString, IsInt } from "class-validator";

export class CreateCatDto {
  @IsString()
  name: string;

  @IsInt()
  age: number;

  @IsString()
  breed: string;
}
```

In the above example, `CreateCatDto` is a DTO that represents the data needed to create a cat. It expects a name and breed of type string, and an age of type number. The `@IsString()` and

@IsInt() decorators are used to apply validation rules.

[↑ Back to Top](#)

1. How can you handle asynchronous operations in NestJS, and what is the role of the Promise object?

NestJS supports asynchronous operations through the use of the `async` and `await` keywords. When a function returns a Promise, it can be awaited, allowing non-blocking execution. The Promise object represents a value that may be available now, or in the future, or never.

```
import { Injectable } from "@nestjs/common";

@Injectable()
export class AppService {
  async getHello(): Promise<string> {
    const result = await someAsyncOperation();
    return `Hello ${result}`;
  }
}
```

In this example, `getHello()` is an asynchronous method that returns a Promise. The `await` keyword is used to pause the execution of the function until `someAsyncOperation()` completes and the Promise is resolved.

Promises are useful for handling single asynchronous operations. If you need to handle a stream of asynchronous values, you might want to use Observables instead, which are provided by the RxJS library and are also integrated into NestJS.

[↑ Back to Top](#)

1. Explain the purpose of the @InjectRepository() decorator in NestJS.

The `@InjectRepository()` decorator in NestJS is used to inject a repository instance into a service or controller. It is commonly used with TypeORM for database interaction, allowing the injection of repositories for specific entities.

A repository in TypeORM is a way to manage entities: it provides methods to insert, update, remove, and load entities. By injecting a repository, you can use these methods in your service or controller.

Here is an example:

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { User } from './user.entity';

@Injectable()
export class UserService {
  constructor(
    @InjectRepository(User)
    private usersRepository: Repository<User>,
  ) {}
}
```

```
findAll(): Promise<User[]> {  
    return this.usersRepository.find();  
}  
}
```

In the above example `@InjectRepository(User)` is used to inject a repository for the `User` entity. This repository is then used in the `findAll` method to load all users.

[↑ Back to Top](#)

1. Explain the purpose of the `@nestjs/jwt` package in NestJS?

The `@nestjs/jwt` package is a NestJS module that provides functionality around JSON Web Tokens (JWTs). JWTs are a popular method for handling authentication and authorization in web applications.

It includes decorators like `@AuthGuard('jwt')` to protect routes with JWT-based authentication and supports the generation and verification of JWTs.

Here are some of the things you can do with the `@nestjs/jwt` package:

Generate JWTs: You can create a new JWT with specific payloads and sign it with a secret key.

Verify JWTs: You can verify a received JWT to ensure it's valid and hasn't been tampered with.

Decode JWTs: You can decode a JWT to access the payload information.

The `@nestjs/jwt` package is often used in combination with the `@nestjs/passport` package, which provides a way to handle authentication in a flexible, modular way. Together, these packages can be used to implement JWT authentication strategy.

[↑ Back to Top](#)

1. Discuss how tokens are used for authorization in an API. What is the difference between authentication and authorization, and how are these processes implemented with tokens?

Tokens, such as JWTs (JSON Web Tokens), are used for authorization in APIs to ensure that a user has permission to access certain resources or perform certain actions.

Authentication is the process of verifying the identity of a user. When a user logs in with their credentials, the server verifies these credentials and if they are valid, the server generates a token. This token often contains information about the user and is sent back to the client.

Authorization on the other hand, is the process of verifying what a user has access to. Once a user is authenticated and their token is sent with each request, the server can verify the token and check if the user has the necessary permissions to perform the requested action.

Here's a basic process:

11. User sends their credentials (like username and password) to the server.
12. Server verifies the credentials. If they're valid, the server generates a token and sends it back to the client.
13. In subsequent requests, the client sends this token in the header of the request.
14. The server verifies the token and checks the user's permissions. If the user has the necessary permissions, the server processes the request.

In this way, tokens serve as proof of authentication and can carry information about user's permissions for authorization. They provide a stateless, scalable solution for securing APIs.

[↑ Back to Top](#)

1. Why is it important for tokens to have an expiration time? How can you implement token expiration in NestJS, and what role do refresh tokens play in maintaining user sessions?

Tokens having an expiration time is important for security reasons. If a token is stolen or leaked, it can be used to gain unauthorized access to the system. By setting an expiration time, you limit the time window in which a stolen token can be used.

In NestJS, you can set the expiration time of a JWT when you sign it using the `JwtService`. Here's an example:

```
this.jwtService.sign(payload, { expiresIn: "60s" });
```

In this example, the token will expire 60 seconds after it's issued.

However, having tokens expire can be inconvenient for the user, as they would have to log in again every time their token expires. This is where `refresh` tokens come in.

[↑ Back to Top](#)

1. Describe the mechanism for a token refresh in NestJS. How can you implement an automatic token refresh strategy to maintain user sessions?

In NestJS, a `token refresh` strategy involves issuing a refresh token when a user logs in.

A refresh token is a special kind of token that can be used to obtain a new access token when the current one expires. When the user logs in, along with the access token, a refresh token is also generated and sent to the client.

When the access token expires, the client sends the refresh token to the server, the server verifies the refresh token and issues a new access token.

This allows the user to stay authenticated without having to log in again, while still limiting the potential damage of a stolen access token.

Refresh tokens usually have a longer expiration time than access tokens, and they can be revoked by the server if needed, for example, in case of a logout.

Here is how you can implement a token refresh strategy:

Issue a Refresh Token: When a user logs in, along with the access token, issue a refresh token. This can be done similarly to how you issue an access token, but typically with a longer expiration time.

Store the Refresh Token: Store the refresh token in your database associated with the user. This allows you to invalidate the refresh token when necessary, such as when the user logs out.

Create a Refresh Endpoint: Create an endpoint in your application that accepts a refresh token and returns a new access token. In this endpoint, you should verify the refresh token, check that it hasn't been invalidated, and then issue a new access token.

Use the Refresh Token: On the client side, when you receive a 401 Unauthorized response, it means the access token has expired. In this case, send a request to the refresh endpoint with the refresh token to get a new access token. Replace the old access token with the new one in your client's storage.

[↑ Back to Top](#)

1. How does NestJS support authentication and authorization?

Nest supports authentication and authorization through various ways:

15.Passport.js Integration: NestJS integrates smoothly with Passport.js, a popular authentication middleware for Node.js. Passport supports a variety of authentication strategies, such as OAuth, JWT, and local (username and password).

16.JWT Module: NestJS provides a JWT module (@nestjs/jwt) for generating and validating JSON Web Tokens, which are commonly used for stateless, server-side authentication.

17.Guards: In NestJS, guards are classes that control whether a given request is allowed to proceed to the route handler. They're often used to implement authorization checks.

18.Decorators: Custom decorators can be used to provide metadata about routes, such as required roles for accessing a route.

19.Interceptors: Interceptors can be used to bind user data to the request based on the provided token.

```
import { Controller, UseGuards, Post, Request } from '@nestjs/common';
import { AuthService } from '../auth/auth.service';
import { LocalAuthGuard } from '../auth/local-auth.guard';

@Controller('auth')
export class AuthController {
  constructor(private authService: AuthService) {}
```

```

    @UseGuards(LocalAuthGuard)
    @Post('login')
    async login(@Request() req) {
        return this.authService.login(req.user);
    }
}

```

In this example, the LocalAuthGuard is a custom guard that uses Passport's local strategy to validate the user's username and password. The login() method then generates a JWT for the authenticated user using the AuthService.

[↑ Back to Top](#)

1. What is the difference between Provider and Services in Nestjs, can we have a provider without an injectable decorator, Give examples?

A **provider** is a more general concept, while a **service** is a specific type of provider. Providers are a fundamental concept in NestJS system of dependency injection. They can be used to inject not only services, but also values, factories, and more.

A **service** is a class annotated with `@Injectable()` decorator, typically used to handle complex business logic or to provide access to shared data. Here's an example:

```

import { Injectable } from "@nestjs/common";

@Injectable()
export class CatsService {
    // ...
}

```

However, a provider doesn't necessarily need to be a service. It can be any value that should be available for injection. For example, you can provide a simple string:

```

{
  provide: 'HelloMessage',
  useValue: 'Hello, World!',
}

```

In this case, `HelloMessage` is a token that can be used to inject the string `Hello, World!`.

While services are typically decorated with `@Injectable()`, other types of providers don't need this decorator. The `@Injectable()` decorator is needed when a class has its own dependencies that need to be injected. If the provider doesn't have any dependencies, like in the string example above, you don't need `@Injectable()`.

[↑ Back to Top](#)

1. What are custom providers and how do they differ from standard Providers in Nest.js?

Providers are essential in NestJS as they form the backbone of the dependency injection system. Their primary role is to create and manage instances of classes that can be injected

into different components, promoting modularity and testability. By using providers, NestJS ensures a robust and organized approach to handling dependencies.

A standard provider in NestJS is typically a class decorated with `@Injectable()`. This class can have dependencies, which are injected through the constructor. Here's an example:

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class CatsService {
  constructor(private readonly catsRepository: CatsRepository) {}
}
```

In this case, `CatsService` is a standard provider that can be injected into other classes, and it has a dependency on `CatsRepository`. But not all providers are services.

A `custom provider` is a more flexible way to provide values for injection, it can also be a simple value or a factory function. And these types of providers don't need the `@Injectable()` decorator. For example, you can have a provider that always provides the number 42:

```
{
  provide: 'MagicNumber',
  useValue: 42,
}
```

In this case, `MagicNumber` is a token that you can use to get the number 42.

`Custom providers` are a way to create providers that aren't just services. They can be values, factory functions, or async factory functions. They give you more flexibility in how you create the things that your app needs.

[↑ Back to Top](#)

1. How can you generate API documentation using Swagger in NestJS? Discuss the importance of documenting your API and how it benefits developers?

Generating API documentation in NestJS can be done using the `@nestjs/swagger` package. This package provides decorators and a `SwaggerModule` to easily create Swagger documentation.

Here is a basic setup:

20. Start by installing the required dependency.

21. `npm install --save @nestjs/swagger`

```
import { NestFactory } from '@nestjs/core';
import { SwaggerModule, DocumentBuilder } from '@nestjs/swagger';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  const config = new DocumentBuilder()
```

```

        .setTitle("Cats example")
        .setDescription("The cats API description")
        .setVersion("1.0")
        .addTag("cats")
        .build();
const document = SwaggerModule.createDocument(app, config);
SwaggerModule.setup("api", app, document);

    await app.listen(3000);
}
bootstrap();

```

In the above example, `SwaggerModule.createDocument(app, config)` generates the Swagger JSON. `SwaggerModule.setup('api', app, document)` serves the Swagger UI at the specified path ('api' in this case).

Documenting your API is important for several reasons:

Ease of Use: It helps other developers understand how to use your API. They can see the available endpoints, the expected request format, and the response format.

Testing: Tools like Swagger UI allow developers to test the API directly from the browser.

Maintenance: It helps maintain the API. When changes are made, the documentation serves as a reference to ensure the API's behavior remains consistent.

Onboarding: It speeds up the process of onboarding new developers. They can quickly understand the API's functionality without needing to dig into the codebase.

[↑ Back to Top](#)

1. Explain the purpose of the `@nestjs/swagger` `ApiProperty()`, `ApiOperation()` decorators?

The `@nestjs/swagger` package provides several decorators to help with creating Swagger documentation for your API. Two of these decorators are `@ApiProperty()` and `@ApiOperation()`.

`@ApiProperty()`: This decorator is used within DTO (Data Transfer Object) classes to provide metadata for properties. This metadata is used to generate the Swagger documentation for the model that the API expects in the request body or returns in the response.

```

import { ApiProperty } from "@nestjs/swagger";

export class CreateUserDto {
  @ApiProperty({
    description: "The id of the user.",
    minimum: 1,
    example: 42,
  })
  id: number;

  @ApiProperty({ description: "The name of the user.", example: "Thomas" })
  username: string;
}

```

```
}
```

The `@ApiProperty` is used to indicate that `id` and `name` are properties of the `CreateUserDto`

`@ApiOperation()`: This decorator is used within controller methods to provide metadata for operations (API endpoints). This metadata is used to generate the Swagger documentation for the operation. It allows developers to provide additional information such as operation summary, description, and custom tags, enhancing the generated Swagger documentation.

Below is an example:

```
import { ApiOperation } from "@nestjs/swagger";

@Controller("users")
export class UsersController {
  @Post()
  @ApiOperation({ summary: "Create user" })
  create(@Body() createUserDto: CreateUserDto) {
    // ...
  }
}
```

`@ApiOperation()` is used to provide a summary for the create operation. This summary will be displayed in the Swagger UI.

There are more decorators that are used to display error messages to the user such as `@ApiNotFoundResponse`, `@ApiBadRequestResponse`, `@ApiInternalServerErrorResponse` and many more. While decorators that display success messages include `@ApiOkResponse`, `@ApiCreatedResponse` etc.

[↑ Back to Top](#)

1. Explain the purpose of the Dockerfile in a NestJS application, and how it facilitates containerization?

A **Dockerfile** is a text document that contains all the commands a user could call on the command line to assemble an image. In the context of a NestJS application, a Dockerfile is used to create a Docker image of the application.

A **Docker image** is a lightweight, standalone, executable package of software that includes everything needed to run an application. It includes code, runtime, system tools, system libraries, and settings.

This image can be run consistently on any machine that has Docker installed, regardless of the underlying operating system.

Here is an example of a Dockerfile for Nest application:

```
// Start from a base image
FROM node:14-alpine // or node:latest to use the latest version of node

// Set the working directory
WORKDIR /usr/src/app
```

```
//Install dependencies
COPY package*.json ./
RUN npm install

// Copy source code
COPY . .

// Expose the application on port 3000
EXPOSE 3000

// Start the application
CMD ["npm", "run", "start"]
```

This Dockerfile does the following:

Starts from a base image with Node.js installed (node:14-alpine).

Sets the working directory in the container to /usr/src/app.

Copies package.json and package-lock.json (if available) to the working directory.

Installs the dependencies using npm install.

Copies the rest of the source code to the working directory.

Exposes port 3000, which the application uses.

Defines the command to start the application (npm run start).

Note: The benefit of this is that it encloses the application and its environment into a single runnable entity (a container). This ensures that the application runs the same way, regardless of where it's deployed, providing consistency and reliability across different deployment environments.

[↑ Back to Top](#)

1. How can you use Docker Compose with NestJS, and what is its role in a multi-container setup?

Docker Compose Docker Compose is a tool for defining and running multi-container applications. Compose simplifies the control of your entire application stack, making it easy to manage services, networks, and volumes in a single, comprehensible YAML configuration file. Then, with a single command, you create and start all the services from your configuration file.

Here's a basic example of a `docker-compose.yml` file for a NestJS application with a PostgreSQL database:

```
version: '3'
services:
  app:
    build: .
    ports:
      - 3000:3000
    depends_on:
      - db
  db:
    image: postgres:13-alpine
    environment:
```

```
POSTGRES_USER: user
POSTGRES_PASSWORD: password
POSTGRES_DB: dbname
```

In this example, there are two services: `app` and `db`. The `app` service is built using the Dockerfile in the current directory, and it exposes port 3000. The `db` service uses the `postgres:13-alpine` image and sets some environment variables to configure the database.

The `depends_on` option is used to express dependency between services, which has two effects:

`db` will be started before `app`. Docker Compose will wait until `db` is "ready" before starting `app`. To start the application with Docker Compose, you would use the command `docker-compose up`.

The benefit of using Docker Compose is that it simplifies the management of multi-container applications. You can start, stop, and rebuild services with a single command, and it ensures that your application's services are started in the correct order.

[↑ Back to Top](#)

1. What is the purpose of the `@nestjs/passport` package, and how does it facilitate authentication in NestJS?

`@nestjs/passport` is a popular node.js authentication library. The main purpose is to facilitate authentication in a NestJS application. It does this by providing a way to implement different authentication strategies (like local, JWT, OAuth, etc.) in a consistent and modular way. It provides a set of tools that make it easier to implement authentication in a NestJS application using Passport.js.

Here is a basic example of how you might use `@nestjs/passport`

```
import { Injectable } from "@nestjs/common";
import { AuthGuard } from "@nestjs/passport";

@Injectable()
export class JwtAuthGuard extends AuthGuard("jwt") {}
```

[↑ Back to Top](#)

1. How can you handle file uploads in NestJS, and what is the role of the Multer library?

NestJS offers convenient ways to handle file uploads, and one common approach is using the `multer` middleware. Additionally, the framework provides the `@UploadedFile` decorator, which simplifies the process of accessing and processing uploaded files in NestJS controllers. These tools collectively offer a flexible and efficient solution for managing file uploads in NestJS applications.

By using the `@UseInterceptors()` decorator with `FileInterceptor` or `FilesInterceptor`, you can handle single or multiple file uploads in your application.

[↑ Back to Top](#)

1. How does NestJS handle database interactions, and what are the supported databases?

NestJS doesn't directly handle database interactions. Instead, it provides integration with several libraries that do, allowing you to choose the one that best fits your needs. Here are some of the most commonly used ones:

TypeORM: This is an Object-Relational Mapping (ORM) library that can be used with a wide variety of databases, including MySQL, PostgreSQL, MongoDB, SQLite, and more. It provides a high-level API for managing database records as JavaScript objects.

Mongoose: If you're working with MongoDB, Mongoose is a great choice. It provides a straight-forward, schema-based solution to model your application data and includes built-in type casting, validation, query building, and business logic hooks.

Sequelize: Sequelize is a promise-based Node.js ORM for Postgres, MySQL, MariaDB, SQLite, and Microsoft SQL Server. It supports the standard CRUD operations, transactions, migrations, and more.

Prisma: Prisma is an open-source database toolkit. It replaces traditional ORMs and can be used to build GraphQL servers, REST APIs, microservices & more.

On how each of these libraries is integrated in nestjs you can visit the [Nest documentation](#)

[↑ Back to Top](#)

1. What is Circular dependency (dependency cycle) in Nestjs, and how can they be fixed?

A **circular dependency** occurs when two classes depend on each other. For example, `class A` needs `class B`, and `class B` also needs `class A`. Circular dependencies can arise in Nest between modules and between providers.

Nest enables resolving circular dependencies between providers in two ways.

1.forward referencing: allows Nest to reference classes which aren't yet defined using the `forwardRef()` utility function. For example, if `CatsService` and `CommonService` depend on each other, both sides of the relationship can use `@Inject()` and the `forwardRef()` utility to resolve the circular dependency. Otherwise Nest won't instantiate them because all of the essential metadata won't be available. Here's an example:

```
import { forwardRef } from '@nestjs/common'

@Injectable()
export class CatsService {
  constructor(
    @Inject(forwardRef(() => CommonService))
    private commonService: CommonService,
  ) {}
}
```

Now let's do the same with CommonService

```
import { forwardRef } from '@nestjs/common'

@Injectable()
export class CommonService {
  constructor(
    @Inject(forwardRef(() => CatsService))
    private catsService: CatsService,
  ) {}
}
```

2.ModuleRef class: An alternative to using `forwardRef()`. for an example you can refactor the above examples and use the `ModuleRef` class to retrieve a provider on one side of the (otherwise) circular relationship.

[↑ Back to Top](#)

1. How can you handle errors in NestJS?

NestJS handles errors using exceptions such as `throw new HttpException()` syntax.

When an exception is thrown, NestJS will automatically send an appropriate HTTP response with a status code and message.

```
import { HttpException, HttpStatus } from '@nestjs/common';

throw new HttpException("Resource not found", HttpStatus.NOT_FOUND);
```

In the above example, an `HttpException` is thrown with a message of 'Resource not found' and a status code of 404 (Not Found).

For more complex error handling, you can create custom exceptions by extending the `HttpException` class. Nestjs has many built-in standard HTTP exceptions that inherit from the base `HttpException` and one can use: These are exposed from the `@nestjs/common` package, and represent many of the most common HTTP exceptions which includes: `BadRequestException`, `UnauthorizedException`, `NotFoundException`, `ForbiddenException`, `NotAcceptableException`, `ConflictException`, `NotImplementedException` and so on.

[↑ Back to Top](#)

1. How does NestJS handle CORS (Cross-Origin Resource Sharing)?

CORS (Cross-Origin Resource Sharing) is a mechanism that allows many resources (e.g., fonts, JavaScript, etc.) on a web page to be requested from another domain outside the domain from which the resource originated.

In the context of web development, an API server runs on a different domain or port from the client-side web application. For security reasons, browsers prohibit web pages from making requests to a different domain than the one the web page came from, unless the server supports CORS.

By enabling CORS on the server, you're allowing the server to respond to cross-origin requests. This means that your server's resources can be accessed from a different domain, protocol, or port than the one your server is hosted on.

NestJS uses the capabilities of the underlying platform (Express or Fastify) to handle **Cross-Origin Resource Sharing (CORS)**. For Express, you can enable CORS globally for all routes in your `main.ts` file like this:

```
import { NestFactory } from "@nestjs/core";
import { AppModule } from "../app.module";

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.enableCors({
    origin: "http://localhost:3000",
    methods: "GET, HEAD, PUT, PATCH, POST, DELETE",
    allowedHeaders: "Content-Type",
  });
  await app.listen(3000);
}
bootstrap();
```

In this example, CORS is enabled only for requests from 'http://localhost:3000' and for the specified methods and headers.

[**↑ Back to Top**](#)

1. Explain the purpose of the ExecutionContext in NestJS Middleware?

`ExecutionContext` can be used to access the `Request` and `Response` objects, or any other details about the current request-response cycle. This can be useful for tasks like logging, validation, transformation, and other operations that need to be performed before the request reaches the route handler.

[**↑ Back to Top**](#)

1. How can you implement soft deletes in NestJS using TypeORM, and why might soft deletes be preferred over hard deletes?

Soft deletes in TypeORM are implemented using the `@DeleteDateColumn` decorator. When you delete an entity that has a `@DeleteDateColumn`, TypeORM doesn't actually remove it from the database. Instead, it sets the `@DeleteDateColumn` to the current timestamp. This is known as a "soft delete".

Here's an example of how you might use `@DeleteDateColumn` in an entity:

```
import { Entity, PrimaryGeneratedColumn, Column, DeleteDateColumn } from 'typeorm';

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;
  @DeleteDateColumn()
  deletedAt: Date;
```

```

    @Column()
    name: string;

    @DeleteDateColumn()
    deletedAt?: Date;
}

```

In this example, when you call `userRepository.softDelete(user.id)`, TypeORM will set `deletedAt` to the current timestamp, but the User will remain in the database.

Soft deletes can be preferred over hard deletes for a few reasons:

1. **Data recovery**: If a record is accidentally deleted, it can be easily restored.

2. **Audit trail**: Soft deletes allow you to keep a history of all records, even ones that are deleted.

3. **Relationship integrity**: If other tables reference the deleted record, those relationships won't be broken by a soft delete.

For information you can read [Nestjs-Query](#)

[↑ Back to Top](#)

1. Explain the concept of environment variables in NestJS, and how can they be utilized for configuration management?

Environment variables are a way to store configuration settings that can change between different environments (like development, staging, production, etc.). They are often used to store sensitive information like database credentials, API keys, or any other configuration that might change depending on the environment.

NestJS provides a `ConfigModule` that uses the `dotenv` package to load environment variables from a `.env` file into `process.env`.

Here's an example of how you might use `ConfigModule` to load environment variables:

```

import { Module } from "@nestjs/common";
import { ConfigModule } from "@nestjs/config";

@Module({
  imports: [ConfigModule.forRoot()],
})
export class AppModule {}

```

In the above example, `ConfigModule.forRoot()` loads the `.env` file and the variables can be accessed anywhere in your application using `process.env`.

[↑ Back to Top](#)

1. What is the role of migration scripts in TypeORM, and how can you create and run migrations in a NestJS application?

Migration scripts in TypeORM are a way to manage changes to your database schema over time. They allow you to version control your database schema and apply updates in a controlled manner. This is especially useful when working in a team or when you need to ensure that your database schema is consistent across different environments (development, staging, production, etc.).

24. First, you need to set up TypeORM in your NestJS application. This typically involves importing the `TypeOrmModule` into your application module and configuring it with your database connection details.

25. After you need to add a `migrations` path and a `cli` configuration to your `ormconfig.json` or `ormconfig.js` file:

```
{
  "type": "postgres",
  "host": "localhost",
  "port": 5432,
  "username": "test",
  "password": "test",
  "database": "test",
  "entities": ["src/**/*.entity.ts"],
  "migrations": ["src/migrations/*.ts"],
  "cli": {
    "migrationsDir": "src/migrations"
  }
}
```

26. To generate a new migration, you can use the TypeORM CLI command `typeorm migration:generate -n MigrationName`. This will create a new migration file in the `src/migrations` directory with a name like `TIMESTAMP-MigrationName.ts`.

27. The generated migration file will have `up` and `down` methods. In the `up` method, you write the SQL to apply the migration, and in the `down` method, you write the SQL to undo the migration.

28. To run the migrations, you can use the TypeORM CLI command `typeorm migration:run`. This will apply all pending migrations in the order they were created.

29. To undo the last migration, you can use the TypeORM CLI command `typeorm migration:revert`. This will run the `down` method of the last applied migration.

[↑ Back to Top](#)

1. What is the purpose of ExecutionContext in NestJS?

`ExecutionContext` represents the context of the currently processed HTTP request. It contains information about the request, response, route handler, and other

details. `ExecutionContext` is often used in custom decorators, guards, and interceptors to access and manipulate request-related information.

[↑ Back to Top](#)

1. What is the purpose of the `@Res()` decorator in NestJS controllers?

Nest provides `@Res()` and `@Response()` decorators. `@Res()` is simply an alias for `@Response()`. `@Res()` or `@Response()` allows you to directly interact with the response object and use its methods.

When using them, you should also import the typings for the underlying library (e.g., `@types/express`) to take full advantage.

Here is an example of using `@Res()` decorator:

```
import { Controller, Get, Res } from "@nestjs/common";
import { Response } from "express";

@Controller("cats")
export class CatsController {
  @Get()
  findAll(@Res() res: Response) {
    res.status(200).send("This action returns all cats");
  }
}
```

Note When you inject either `@Res()` or `@Response()` in a method handler, you put Nest into Library-specific mode for that handler, and you become responsible for managing the response. When doing so, you must issue some kind of response by making a call on the response object (e.g., `res.json(...)` or `res.send(...)`), or the HTTP server will hang.

[↑ Back to Top](#)

1. Explain the various Modules in NestJS?

A `module` is a class annotated with a `@Module()` decorator. The `@Module()` decorator provides metadata that Nest makes use of to organize the application structure.

`modules` are a fundamental aspect of the framework's architecture. They help organize the application into logical and manageable sections. There are three main types of modules in NestJS:

30.Feature Modules: These are the most common type of modules and are used to group related features together. They keep the code organized and establish clear boundaries. This helps us manage complexity and develop with SOLID principles, especially as the size of the application and/or team grow.

For example: The `CatsController` and `CatsService` belong to the same application domain. As they are closely related, it makes sense to move them into a feature module.

```
import { Module } from "@nestjs/common";
import { CatsController } from "../cats.controller";
import { CatsService } from "../cats.service";
```

```
@Module({
  controllers: [CatsController],
  providers: [CatsService],
})
export class CatsModule {}
```

31.Shared Modules: modules are singletons by default, and thus you can share the same instance of any provider between multiple modules effortlessly. When a module is imported into another module, all of its providers are made available to the importing module. Therefore, any module that provides shared functionality should be imported wherever that functionality is needed.

Let's imagine that we want to share an instance of the `CatsService` between several other modules. In order to do that, we first need to export the `CatsService` provider by adding it to the module's `exports` array, as shown below:

```
import { Module } from "@nestjs/common";
import { CatsController } from "../cats.controller";
import { CatsService } from "../cats.service";

@Module({
  controllers: [CatsController],
  providers: [CatsService],
  exports: [CatsService],
})
export class CatsModule {}
```

Now any module that imports the `CatsModule` has access to the `CatsService` and will share the same instance with all other modules that import it as well.

32.Global modules: When you want to provide a set of providers which should be available everywhere out-of-the-box (e.g., helpers, database connections, etc.), make the module global with the `@Global()` decorator.

```
import { Module, Global } from "@nestjs/common";
import { CatsController } from "../cats.controller";
import { CatsService } from "../cats.service";

@Global()
@Module({
  controllers: [CatsController],
  providers: [CatsService],
  exports: [CatsService],
})
export class CatsModule {}
```

The `@Global()` decorator makes the module global-scoped. Global modules should be registered only once, generally by the root or core module. In the above example, the `CatsService` provider will be present, and modules that wish to inject the service will not need to import the `CatsModule` in their imports array.

33.Dynamic Modules: enables you to easily create customizable modules that can register and configure providers dynamically. They are created using the

`register()` method, which takes an options object and returns a dynamic module.

[**↑ Back to Top**](#)

1. How can you secure your NestJS application?

Securing a NestJS application involves several aspects, including authentication, authorization, data validation, and error handling. Here are some ways to secure your NestJS application:

Authentication: You can use `Passport.js`, a popular authentication library, which is integrated into NestJS via the `@nestjs/passport` module. Passport supports a wide range of authentication strategies, including OAuth, JWT, and local username/password.

Authorization: NestJS provides the `@Roles()` decorator and `AuthGuard` to handle role-based access control. You can define roles on your route handlers and then use `AuthGuard` to check if the authenticated user has the required roles.

Data Validation: Use `class-validator` and `class-transformer` along with the `ValidationPipe` provided by NestJS to validate incoming request data. This can help prevent common web vulnerabilities like SQL injection and cross-site scripting (XSS).

Error Handling: Use filters to handle exceptions. NestJS provides the `@Catch()` decorator to create exception filters that can catch exceptions and return a user-friendly error response.

Rate Limiting: Use the `@nestjs/throttler` package to limit the number of requests a client can make to your API in a given amount of time.

HTTPS: Use HTTPS to encrypt data in transit between the client and your server. This can be done by providing SSL certificates to the `NestFactory.create()` method.

Helmet: Use Helmet, a collection of middleware functions that set HTTP headers to help protect your application from some well-known web vulnerabilities. NestJS provides the `@nestjs/platform-express` package which includes support for Helmet.

CORS: Configure Cross-Origin Resource Sharing (CORS) properly to restrict which domains can access your API. This can be done using the `enableCors()` method on the application instance.

Remember, security is a broad and complex topic, and these are just some of the ways to secure your NestJS application.

[**↑ Back to Top**](#)

1. What is the entry file of NestJs application?

The entry file of a NestJS application is typically `main.ts`. This is where the application's root module is bootstrapped, which kickstarts the application.

Here's an example of what the `main.ts` file might look like:

```
import { NestFactory } from "@nestjs/core";
import { AppModule } from "../app.module";
```



```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(3000);
}
bootstrap();
```

`NestFactory.create(AppModule)` initializes the application with the root module (`AppModule`), and `app.listen(3000)` starts the application, listening for incoming requests on port 3000.

[**↑ Back to Top**](#)

1. What is the difference between dependency injection and inversion of control (IoC)?

Dependency Injection (DI) and Inversion of Control (IoC) are both design patterns used to reduce the coupling between classes, making the code more modular, easier to test and maintain. However, they are not the same thing, but rather, DI is a form of IoC.

Inversion of Control (IoC) is a general principle where the control flow of a program is inverted: instead of the programmer controlling the flow of a program, the external framework or runtime controls it.

Dependency Injection (DI) is a form of IoC where the creation and binding of dependent objects is controlled by a container or a framework. Instead of a class creating or finding its dependencies, they are passed in (injected) at runtime by another piece of code, typically a container or a framework. This makes the code more flexible, testable and modular because it decouples the usage of an object from its creation. This is the form of IoC that is used in many modern frameworks such as Angular, Spring, and NestJS.

In summary IoC is a design principle which can be implemented in several ways, one of which is DI

[**↑ Back to Top**](#)

1. How can you implement Caching in NestJS?

Caching is a great and simple technique that helps improve your app's performance. It acts as a temporary data store providing high performance data access.

NestJS supports caching through various mechanisms, including the use of caching libraries like `cache-manager` and built-in decorators such as `@CacheKey` and `@CacheTTL`. By incorporating caching strategies in your application, you can enhance performance and reduce response times for frequently requested data.

In order to enable caching, import the `CacheModule` and call its `register()` method

```
import { Module } from "@nestjs/common";
import { CacheModule } from "@nestjs/cache-manager";
import { AppController } from "../app.controller";

@Module({
```

```
imports: [CacheModule.register()],
controllers: [AppController],
})
export class AppModule {}
```

To interact with the cache manager instance, inject it to your class using the `CACHE_MANAGER` token, as follows:

```
constructor(@Inject(CACHE_MANAGER) private cacheManager: Cache) {}
```

The `get` method is used to retrieve items from the cache. If the item does not exist in the cache, `null` will be returned.

```
const value = await this.cacheManager.get("key");
```

To add an item to the cache, use the `set` method:

```
await this.cacheManager.set("key", "value");
```

The default expiration time of the cache is 5 seconds, however you can change this.

```
await this.cacheManager.set("key", "value", 1000);
```

To disable expiration of the cache, set the `t t l` configuration property to 0:

```
await this.cacheManager.set("key", "value", 0);
```

To remove an item from the cache, use the `del` method:

```
await this.cacheManager.del("key");
```

To clear the entire cache, use the `reset` method:

```
await this.cacheManager.reset();
```

[↑ Back to Top](#)

1. Explain the purpose of the Dependency Inversion Principle (DIP) in NestJS?

The Dependency Inversion Principle (DIP) is one of the five principles of [SOLID](#), an acronym. The principle states that:

1. High-level modules should not depend on low-level modules. Both should depend on abstractions.
2. Abstractions should not depend on details. Details should depend on abstractions.

In the context of NestJS, or any other framework that supports dependency injection, the purpose of DIP is to reduce the `coupling` between modules, making the system more flexible, easier to test, and easier to maintain.

By depending on `abstractions`, not on concrete implementations, you can easily swap out modules without changing the high-level code. For example, you might have a service that depends on a repository. If you code to an interface, you can easily change the repository (e.g., from an in-memory repository to a database repository) without changing

the service code.

For more information about DIP you can check [here](#)

NestJS supports DIP through its modular system and the use of decorators like `@Injectable()`, `@Inject()`, and custom providers. These features allow you to define providers and inject them where needed, making it easy to manage dependencies and adhere to the Dependency Inversion Principle.

[↑ Back to Top](#)

1. How can you schedule tasks in NestJS?

Task scheduling allows you to schedule arbitrary code (methods/functions) to execute at a fixed date/time, at recurring intervals, or once after a specified interval.

Nest provides the `@nestjs/schedule` package, which integrates with the popular `Node.js cron` package.

34.To implement scheduling, you can install the required dependencies.

```
npm install --save @nestjs/schedule
```

35.Then, import the `ScheduleModule` into your module:

```
import { Module } from "@nestjs/common";
import { ScheduleModule } from "@nestjs/schedule";
import { TasksService } from "../tasks.service";

@Module({
  imports: [ScheduleModule.forRoot()],
  providers: [TasksService],
})
export class TasksModule {}
```

36.Now, you can use the decorators in your service to schedule tasks. Here's an example:

```
import { Injectable } from "@nestjs/common";
import { Cron, CronExpression } from "@nestjs/schedule";

@Injectable()
export class TasksService {
  @Cron(CronExpression.EVERY_5_SECONDS)
  handleCron() {
    console.log("Called every 5 seconds");
  }
}
```

In the above example, `handleCron()` will be called every 5 seconds. The `@Cron()` decorator takes a `CronExpression` which determines the schedule.

Remember, the `ScheduleModule` uses the `node-schedule` package under the hood, so you can use any cron expression that `node-schedule` supports.

[↑ Back to Top](#)

1. How can you handle database transactions in NestJS, and why are transactions important in certain scenarios?

Database transactions in NestJS can be handled using the `TypeORM` package. Transactions are important when you want to ensure data integrity. If a series of database operations need to succeed or fail together, transactions can ensure that if any operation fails, all changes are rolled back and the database remains in a consistent state.

This is particularly important in scenarios such as financial operations, where it's crucial that either all parts of a transaction are completed or none of them are.

[↑ Back to Top](#)

1. How can you implement versioning in NestJS APIs?

Versioning allows you to have different versions of your controllers or individual routes running within the same application.

There are 4 types of versioning that are supported:

37.**URI Versioning**: The version will be passed within the URI of the request (default).

38.**Header Versioning**: A custom request header will specify the version.

39.**Media Type Versioning**: The Accept header of the request will specify the version.

40.**Custom Versioning**: Any aspect of the request may be used to specify the version(s). A custom function is provided to extract said version(s).

To enable **Header Versioning** for your application, do the following:

```
const app = await NestFactory.create(AppModule);
app.enableVersioning({
  type: VersioningType.HEADER,
  header: "Custom-Header",
});
await app.listen(3000);
```

Check [this](#) to see how the above types of versioning are implemented.

[↑ Back to Top](#)

1. Explain the purpose of the `@nestjs/graphql` Resolver and `@nestjs/graphql` Scalar decorators, and how they relate to GraphQL in NestJS?

GraphQL is a powerful query language for APIs and a runtime for fulfilling those queries with your existing data. It's an elegant approach that solves many problems typically found with REST APIs.

The `@nestjs/graphql` package provides decorators that allow you to define GraphQL schemas directly from your TypeScript classes.

41. `@Resolver()` : This decorator is used to mark a class as a GraphQL resolver. Resolvers are the building blocks of GraphQL servers. They are responsible for fetching the data for individual fields in a schema. When you send a query to a GraphQL server, the server uses resolver functions to produce a response. On how its used check [here](#)
42. `@Scalar()` : This decorator is used to define a custom scalar. Scalars are primitive values: Int, Float, String, Boolean, or ID. When you need to use a custom primitive type (like a Date or a type from your database), you can define a custom scalar. On how its used check [here](#)

Read [this](#) to understand the difference between GraphQL and REST

[↑ Back to Top](#)

1. Explain the concept of Serialization and Deserialization in NestJS?

Serialization and deserialization are fundamental concepts in computer science, not just in NestJS. They are used when data needs to be converted into a format that can be stored or transmitted and then reconstructed later.

Serialization: This is the process of converting a data structure or object state into a format that can be stored (for example, in a file or memory buffer) or transmitted (for example, across a network connection link) and reconstructed later (possibly in a different computer environment). For more about serialization you can check [here](#).

Deserialization: This is the reverse process of serialization, where the serialized format is converted back into an actual object in memory.

In the context of NestJS, these concepts are often used when dealing with HTTP requests and responses. For example, when you send data from a client to a NestJS server, the data is serialized into a JSON format, transmitted over the network, and then deserialized back into a JavaScript object on the server.

NestJS provides a Pipes mechanism that can be used for data transformation (serialization/deserialization) and validation. For example, the `ValidationPipe` provided by NestJS can be used to automatically validate and transform incoming request payloads into instances of DTO classes.

[↑ Back to Top](#)

1. Explain the role of NestJS middleware in the context of Microservices and provide a scenario where middleware is beneficial in a Microservices setup?

Middleware is a function that is executed before the route handler. Middleware functions have access to the request and response objects, and the `next()` middleware function in the application's request-response cycle. They can execute any code, make changes to the request and the response objects, end the request-response cycle, and call the next middleware function in the stack.

In the context of microservices, middleware can play several important roles:

- 43. **Request Logging:** Middleware can be used to log details of incoming requests. This can be useful for debugging, as well as for tracking usage patterns and user behavior.
- 44. **Authentication and Authorization:** Middleware can verify a user's identity and permissions before a request reaches a service. This can prevent unauthorized access and ensure that each service doesn't have to implement its own authentication checks.
- 45. **Error Handling:** Middleware can catch and handle errors. This can help to ensure that the microservice responds with a well-defined error structure even when something goes wrong.
- 46. **Rate Limiting:** Middleware can track the number of requests from a client and limit usage to prevent abuse.

[**↑ Back to Top**](#)

1. Discuss the different types of coupling, such as tight coupling and loose coupling, and provide examples of how NestJS modules contribute to achieving loose coupling in a modularized application.

Coupling is the degree of interdependence between software modules, a measure of how closely connected two routines or modules are, the strength of the relationships between modules.

Tight Coupling: In this case, a module (or class) is highly dependent on another module. Changes in one module may require changes in the dependent module. This makes the system harder to maintain and evolve over time.

Loose Coupling: Here, a module is not highly dependent on other modules. Changes in one module have minimal or no effect on other modules. This makes the system more maintainable and adaptable to change.

NestJS promotes loose coupling through its modular development structure. Each module contains a portion of the application's functionality and can operate independently of other modules. This means that changes in one module do not affect others, leading to a loosely coupled system.

Here is a more easier example:

```
// users.service.ts
import { Injectable } from '@nestjs/common';
import { User } from '../user.entity';

@Injectable()
export class UsersService {
  private users: User[] = [];

  create(user: User) {
    this.users.push(user);
  }
}
```

```

    }

    findAll(): User[] {
        return this.users;
    }
}

// meal.service.ts
import { Injectable } from '@nestjs/common';
import { UsersService } from '../users/users.service';

@Injectable()
export class MealService {
    constructor(private usersService: UsersService) {}

    createMeal(userId: string, MealData: CreateMealDTO) {
        const user = this.usersService.findById(userId);
        // Create meal for the user
    }
}

```

In above example, MealService depends on UsersService, but it doesn't manipulate user data directly. This is an example of loose coupling.

[↑ Back to Top](#)

1. How does NestJS support Server-Sent Events (SSE), and what are the primary advantages of using SSE for real-time communication in web applications?

Server-Sent Events (SSE) is a server push technology enabling a client to receive automatic updates from a server via HTTP connection. SSE is a one-way communication channel from server to client. If you need bi-directional communication, you might want to use WebSockets instead.

SSE they have been used in Facebook/Twitter updates, stock price updates, news feeds etc.

To enable Server-Sent events on a route (route registered within a controller class), annotate the method handler with the `@Sse()` decorator.

```

@sse('sse')
sse(): Observable<MessageEvent> {
    return interval(1000).pipe(map((_) => ({ data: { hello:
'world' } } })));
}

```

In the example above, we defined a route named SSE that will allow us to propagate real-time updates. These events can be listened to using the [EventSource API](#).

Server-Sent Events (SSE) have several advantages for real-time communication in web applications:

1. **Built on HTTP:** SSE is built on HTTP, which makes it compatible with most firewalls and networks without requiring any special configuration.

2. **Automatic Reconnection:** If a connection is lost, the browser will automatically try to reconnect to the server.

3.Event IDs: The server can send an ID with each event, so if a client gets disconnected, it can reconnect and get all the events it missed.

4.Efficient Updates: SSE is ideal for applications that require real-time updates from the server (like live news updates, real-time analytics, etc.). The server can push updates to the client as soon as new data is available.

For more detailed information about SSE kindly refer to [this](#)

[**↑ Back to Top**](#)